

Universidad del Valle de Guatemala

CC2008 - Introducción a la Programación Orientada a Objetos

Semestre II, 2026

Polimorfismo vía herencia

Defensa de QTZ2

Josue David Garcia Mendez 26375

1. Planteamiento del problema

El satélite Quetzal-2 tiene tres tipos de módulos que comparten recursos:

- **Módulos de energía** (paneles solares): generan energía.
- **Módulos de vuelo** (cámaras o sensores en órbita): consumen energía y recolectan datos científicos.
- **Módulos de tierra** (antenas en el campus): consumen energía y descargan los datos que están en órbita.

El operador necesita consultar los módulos y avanzar la simulación ciclo por ciclo, sin perder lo acumulado y sin cambiar el orden en que participan.

Objetivo general

Diseñar un programa orientado a objetos que simule el funcionamiento de los módulos uno tras otro y mantenga resultados coherentes. Debe usar herencia, polimorfismo, encapsulación y el patrón MVC (separar el modelo, la consola y el control de las solicitudes).

Objetivos específicos

1. Poner los datos comunes en una superclase abstracta y el comportamiento propio de cada tipo en tres subclases.
2. Definir cómo se procesa un ciclo: recursos compartidos, cambios inmediatos y condiciones que se puedan comprobar.
3. Resolver las opciones del menú, incluyendo el catálogo por costo y el resumen de comunicaciones.
4. Definir una carga inicial de diez módulos con datos históricos coherentes y casos de prueba para verificar el programa.

Alcance de este documento

Tiene como propósito cubrir la herencia, polimorfismo y separación de responsabilidades con MVC. También incluye el diagrama UML.

2. Requisitos del sistema

Usuario: el operador de la misión, que usa el programa por consola. La misión ya viene cargada al iniciar.

Estado que se administra: los módulos, la energía disponible, los MB pendientes en órbita, los MB descargados en total y el número de ciclos transcurridos.

Código	Requisito
--------	-----------

RF01	Cargar una misión en progreso con al menos 10 módulos de los tres tipos, en una sola lista List<Modulo>, conservando el orden de carga.
RF02	Listar todos los módulos con sus datos comunes, sus datos propios y sus acumulados.
RF03	Buscar por ID o por nombre y mostrar la información completa de lo encontrado. Avisar si no hay resultados.
RF04	Mostrar el catálogo de menor a mayor costo de construcción, sin cambiar el orden en que operan los módulos.
RF05	Avanzar un ciclo: dar una oportunidad a cada módulo, aplicar sus cambios al instante, informar qué hizo cada uno (o por qué no pudo operar) y mostrar los recursos finales.
RF06	Mostrar el resumen de comunicaciones: total de módulos de tierra, cuántos están activos, capacidad conjunta de los activos, total histórico descargado y todos los módulos empatados en el máximo.
RF07	Conservar recursos y acumulados entre ciclos. Las consultas no modifican la simulación.
RF08	Permitir salir del programa y recuperarse de entradas inválidas en el menú.

Restricciones de diseño

- Se programa en Java con el patrón MVC.
- Los tres tipos de módulos se guardan como referencias de tipo Modulo.
- El comportamiento de cada ciclo se resuelve sobrescribiendo un método en las subclases.
- Las capacidades y consumos se fijan al crear el objeto y no cambian durante la misión.

3. Reglas de negocio

Cuándo y cómo opera cada módulo

Tipo	Condiciones para operar	Qué pasa cuando opera
Energía	Estar activo.	Suma su generación a la energía disponible y sube en 1 sus ciclos acumulados.
Vuelo	Estar activo y que la energía disponible sea mayor o igual a su consumo.	Resta su consumo completo, suma todos sus MB por ciclo a los datos en

		órbita y sube en 1 sus ciclos acumulados.
Tierra	Estar activo, que haya MB pendientes en órbita y que la energía disponible sea mayor o igual a su consumo.	Descarga el menor valor entre su capacidad y los MB pendientes, consume su energía completa y actualiza los MB en órbita y los dos históricos de descarga (el del módulo y el de la misión).

Si un módulo no puede operar, no cambia ningún recurso ni acumulado. Se informa la causa y el ciclo continúa con el siguiente.

Orden de comprobación

- Vuelo: primero si está activo, luego si hay energía.
- Tierra: primero si está activa, luego si hay datos, luego si hay energía.
- Energía: solo se comprueba si está activo.

Secuencia del ciclo

- Cada módulo participa una sola vez por ciclo, en el orden de carga. Si no pudo operar, no vuelve a intentarlo en ese ciclo.
- Los cambios se ven de inmediato: cada módulo usa los recursos que dejó el anterior.
- El contador de ciclos de la misión sube en 1 al terminar el recorrido, aunque ningún módulo haya operado.
- Los ciclos acumulados de vuelo y energía solo suben cuando el módulo realmente opera.
- La energía y los datos sobrantes pasan al siguiente ciclo.
- La energía es un saldo numérico, sin máximo ni batería. Los datos en órbita son una reserva común: no se asigna cada descarga a una cámara o sensor en particular.

Unidades y validaciones numéricas

- MB, energía y costo se representan con long (números enteros). El costo se da en "unidades monetarias de referencia", sin indicar moneda.
- Capacidades y consumos deben ser positivos. Costos, recursos e históricos deben ser no negativos.
- Antes de cualquier cambio se verifica que el resultado no exceda el máximo que cabe en un long. Si lo excediera, la operación se rechaza y el estado no se modifica.

Identidad y búsquedas

- Los IDs son únicos. Para compararlos se quitan los espacios de los extremos y se ignoran las mayúsculas.

- Los nombres pueden repetirse. La búsqueda por nombre es exacta (ignorando mayúsculas y espacios de los extremos) y devuelve todas las coincidencias.

Salud

La salud es un número entero de 0 a 100. Al cargar la misión se rechaza un módulo activo con salud 0. Fuera de esa validación, la salud solo se muestra: no cambia lo que el módulo produce o consume. No hay desgaste. Los datos constantes y el ID no tienen setters públicos.

4. Organización del programa (MVC)

- **Modelo:** guarda el estado y las reglas de la misión.
- **Vista:** lee y escribe en la consola.
- **Controlador:** interpreta la opción elegida y coordina al modelo y a la vista.

Con esto, la vista se puede cambiar sin afectar las reglas.

Componente	Clases	Responsabilidad
Modelo (módulos)	Modulo, ModuloVuelo, ModuloTierra, ModuloEnergia	Representar los componentes y ejecutar el comportamiento propio de cada uno.
Modelo (misión)	Mision, RecursosMision	Administrar la lista de módulos, las consultas, el ciclo y los cambios válidos de recursos.
Resultados	ResultadoOperacion, ResumenComunicaciones	Llevar datos del modelo hacia la vista.
Vista	VistaConsola	Leer entradas, mostrar el menú, los módulos, los resultados y los mensajes.
Controlador	Controlador	Relacionar cada opción del menú con una operación del modelo y un método de la vista.
Inicio	Main	Crear la carga inicial, conectar los componentes y arrancar el menú.
Apoyo	Tipoinstrumento (enumeración)	Limitar el instrumento de vuelo a CAMARA o SENSOR.

Las clases que se usan son aprox 11 clases y 1 enumeración. Ocho clases forman el núcleo de MVC (las cuatro de módulos, Mision, RecursosMision, VistaConsola y Controlador) y tres son de apoyo (ResultadoOperacion, ResumenComunicaciones y Main). No hay clases separadas para cámara y sensor porque siguen las mismas reglas y solo cambia el tipo de instrumento.

Cómo se reparte el trabajo en un ciclo

1. VistaConsola entrega la opción elegida a Controlador.
2. Controlador llama a Mision.avanzarCiclo().
3. Mision recorre su lista y llama a procesarCiclo(recursos) en cada Modulo.
4. Cada subclase consulta y modifica RecursosMision y devuelve un ResultadoOperacion.
5. Controlador entrega los resultados y los recursos finales a la vista para mostrarlos.

Límites entre componentes

- Los módulos y la misión no usan Scanner ni System.out.
- La vista no decide si alcanza la energía y no modifica recursos.
- El controlador no repite el algoritmo de cada subclase.
- Main solo prepara los objetos. Las validaciones las hacen los constructores y el modelo.

Opciones del menú

Opción	Acción	Método del controlador
1	Listar módulos	listarModulos()
2	Buscar módulo	buscarModulo()
3	Catálogo por costo	mostrarCatalogo()
4	Avanzar un ciclo	avanzarCiclo()
5	Resumen de comunicaciones	mostrarComunicaciones()
0	Salir	termina el bucle del menú

5. Especificación de las clases

5.1 Clase abstracta Modulo

Guarda los datos comunes. No se puede instanciar porque un módulo genérico no tiene una operación concreta. Su constructor es protected, recibe los cinco datos comunes y cada subclase se los pasa con super. Todos los atributos son private.

Atributo	Tipo	Condición
----------	------	-----------

id	String	Único, no vacío y constante. Se limpia al construir (sin espacios en los extremos, en mayúsculas).
nombre	String	No vacío.
salud	int	Entre 0 y 100.
activo	boolean	Indica si participa en los ciclos.
costoConstruccion	long	No negativo y constante.

Métodos

Método	Descripción
abstract ResultadoOperacion procesarCiclo(RecursosMision recursos)	Cada subclase lo implementa con su propia regla. Recibe la misma instancia de recursos que usan los demás módulos (no puede ser null). Un impedimento normal (inactivo, sin energía, sin datos) se devuelve como resultado y no interrumpe el recorrido.
String toString()	Sobrescribe el de Object. Devuelve los datos comunes como texto (no imprime). Las subclases lo sobrescriben, usan super.toString() y agregan sus datos.
final boolean equals(Object otro)	Dos módulos son iguales si tienen el mismo ID. null u objetos que no sean Modulo nunca son iguales.
final int hashCode()	Coherente con equals: se calcula solo con el ID.
getId(), getNombre(), getSalud(), isActive(), getCostoConstruccion()	Solo lectura. No hay setters generales porque el menú no permite cambiar nombre, salud ni actividad.

5.2 Enumeración TipoInstrumento

Tiene dos valores: CAMARA y SENSOR.

5.3 Clase ModuloVuelo (hereda de Modulo)

Instrumento que gasta energía para recolectar datos científicos. Su constructor recibe los datos comunes más los propios.

Atributo propio	Tipo	Condición
tipoInstrumento	TipInstrumento	CAMARA o SENSOR; constante, no nulo.
datosPorCiclo	long	MB que recolecta por operación; positivo y constante.
consumoEnergia	long	Energía que gasta por operación; positiva y constante.
ciclosAcumulados	long	Veces que ha recolectado; no negativo.

Método	Descripción
procesarCiclo(recursos) (@Override)	Comprueba si está activo y si hay energía. Si puede operar, pide a RecursosMision la recolección completa y sube ciclosAcumulados. No hay recolecciones parciales.
getTotalRecolectado()	datosPorCiclo × ciclosAcumulados. Incluye lo ya descargado: descargar no reduce este histórico.
toString() (@Override)	Datos comunes, instrumento, MB por ciclo, consumo, ciclos y total recolectado.
Getters	De sus cuatro atributos propios. Sin setters: ciclosAcumulados solo cambia cuando el módulo opera con éxito.

5.4 Clase ModuloEnergia (hereda de Modulo)

Generador solar.

Atributo propio	Tipo	Condición
energiaPorCiclo	long	Energía que genera por operación; positiva y constante.
ciclosAcumulados	long	Veces que ha generado; no negativo.

Método	Descripción
--------	-------------

procesarCiclo(recursos) (@Override)	Si está activo, suma energiaPorCiclo a los recursos y sube ciclosAcumulados. Si está inactivo, devuelve la causa sin cambiar nada.
getTotalGenerado()	energiaPorCiclo × ciclosAcumulados. Lo que otros módulos consuman después no lo reduce.
toString() (@Override)	Datos comunes, generación por ciclo, ciclos y total generado.
getEnergiaPorCiclo(), getCiclosAcumulados()	Solo lectura.

El total de vuelo y el de energía no se guardan si no que se calculan multiplicando un valor constante por los ciclos. Así no existen dos versiones del mismo dato que puedan sobrescribirse de alguna manera.

5.5 Clase ModuloTierra (hereda de Modulo)

Antena que recibe datos desde la reserva en órbita.

Atributo propio	Tipo	Significado
estacion	String	Estación a la que pertenece la antena; no vacía.
capacidadDescarga	long	Máximo de MB por descarga; positivo y constante.
energiaPorDescarga	long	Energía completa que consume por descarga; positiva y constante.
totalDescargado	long	MB históricos recibidos por esta antena; no negativo.

Método	Descripción
procesarCiclo(recursos) (@Override)	Comprueba actividad, datos pendientes y energía (en ese orden). Descarga hasta su capacidad, consume la energía completa y suma los MB descargados a totalDescargado.
toString() (@Override)	Datos comunes, estación, capacidad, consumo e histórico. La estación se incluye porque el informe de comunicaciones la pide.
Getters	getEstacion(), getCapacidadDescarga(), getEnergiaPorDescarga(), getTotalDescargado().

Aquí el histórico **sí se guarda**, porque no se puede calcular con una capacidad fija: una descarga puede ser menor que la capacidad si quedan pocos datos.

5.6 Clase RecursosMision

Guarda los saldos globales de la misión en tres campos private de tipo long: energiaDisponible, datosEnOrbita y totalDescargado. Es el **único lugar** donde existen estos saldos. Todos los módulos reciben una referencia a la misma instancia. El constructor valida los saldos iniciales.

Método	Descripción
boolean hayEnergia(long cantidad)	Dice si el saldo alcanza para una cantidad positiva, sin modificarlo.
void generarEnergia(long cantidad)	Suma una cantidad positiva, después de validar que el resultado cabe en un long.
boolean intentarRecolectar(long consumo, long datos)	Si falta energía, devuelve false y no cambia nada. Si alcanza, resta el consumo y suma los datos a la reserva en una sola operación.
long intentarDescargar(long consumo, long capacidad)	Si falta energía o no hay datos, devuelve 0 sin cambios. Si no, devuelve los MB descargados y actualiza juntos la energía, los datos en órbita y el histórico global.
getEnergiaDisponible(), getDatosEnOrbita(), getTotalDescargado()	Solo lectura. No hay setters.

Los valores inválidos lanzan IllegalArgumentException. Antes de cada cambio se verifican todos los cálculos, para no dejar actualizaciones a medias.

5.7 Clase Mision

Administra los módulos, avanza el tiempo y resuelve las consultas. Su constructor recibe List<Modulo> modulos, RecursosMision recursos y long ciclosTranscurridos. Hace una **copia de la lista** (para que nadie desde afuera pueda modificarla), conserva su orden y verifica: IDs únicos, que estén los tres tipos, al menos 10 módulos y que los acumulados sean coherentes.

Atributo	Tipo	Responsabilidad
modulos	List<Modulo>	Única colección principal de módulos; privada y final.
recursos	RecursosMision	Estado global compartido; privado y final.
ciclosTranscurridos	long	Ciclos completos de la misión, incluyendo los anteriores a la carga.

Método	Descripción
List<Modulo> listarModulos()	Copia no modificable en el orden de carga.
Optional<Modulo> buscarPorId(String id)	Limpia el ID y devuelve el módulo o Optional.empty(). Un texto vacío no tiene coincidencias.
List<Modulo> buscarPorNombre(String nombre)	Todas las coincidencias exactas, en el orden original; lista vacía si no hay.
List<Modulo> obtenerCatalogoPorCosto()	Ordena una copia temporal por costo ascendente. En empates, ordena por ID. Nunca ordena la lista operativa.
List<ResultadoOperacion> avanzarCiclo()	Llama a procesarCiclo(recursos) una vez por módulo, guarda los resultados en orden y sube ciclosTranscurridos al final. Un impedimento normal no aborta el ciclo.
ResumenComunicaciones obtenerResumenComunicaciones()	Filtra los módulos de tierra y calcula cantidades, capacidad activa, histórico y todos los máximos. No modifica recursos ni guarda otra lista de antenas.
RecursosMision getRecursos()	Permite consultar el estado final para mostrarlo. El controlador y la vista solo lo leen.
long getCiclosTranscurridos()	Número de ciclos completos.

Las listas de búsqueda, catálogo y resumen son **temporales**: no reemplazan a la lista principal. Para las consultas específicas de tierra se puede identificar ModuloTierra al recorrer la lista, pero el ciclo **nunca** usa una cadena de instanceof para decidir qué hacer.

5.8 Clase ResultadoOperacion

Describe lo que hizo (o no pudo hacer) un módulo en un ciclo. Sus campos son private final, se asignan en el constructor y se leen con getters. No imprime nada. Las cantidades son del ciclo actual, no totales históricos.

Atributo	Tipo	Contenido
idModulo, nombreModulo	String	Qué módulo participó.

opero	boolean	true solo si realizó su operación.
motivo	String	Código: OPERACION_REALIZADA, INACTIVO, ENERGIA_INSUFICIENTE o SIN_DATOS.
energiaGenerada	long	Energía que agregó este módulo.
energiaConsumida	long	Energía que descontó este módulo.
datosRecolectados	long	MB que agregó a la reserva en órbita.
datosDescargados	long	MB que pasaron de órbita a tierra.

Las cantidades no son negativas. Si opero es false, todas valen cero. El constructor valida que motivo sea uno de los códigos permitidos y que concuerde con opero. La vista convierte los códigos en textos legibles.

5.9 Clase ResumenComunicaciones

Agrupar el resultado de la consulta de comunicaciones. Mision lo construye y VistaConsola lo muestra de inmediato. No tiene setters y se vuelve a calcular en cada consulta, para que no quede desactualizado después de un ciclo.

Atributo	Tipo	Contenido
totalModulosTierra	int	Antenas totales (activas o no).
modulosTierraActivos	int	Antenas activas.
capacidadActivaTotal	long	Suma de capacidades de las antenas activas (MB por ciclo).
totalHistoricoDescargado	long	Suma de los históricos de todas las antenas.
mayoresDescargas	List<ModuloTierra>	Lista no modificable con todas las antenas empatadas en el máximo histórico.

Si todos los históricos son cero, todas las antenas empatan. Si no hubiera antenas, las cantidades serían cero y la lista quedaría vacía (la carga obligatoria sí incluye módulos de tierra).

5.10 Clase VistaConsola

Tiene un Scanner privado. **Toda** la salida por consola ocurre aquí. No guarda una referencia permanente a Mision ni modifica objetos del modelo.

Método	Descripción
int leerOpcionMenu()	Muestra las opciones 1 a 5 y 0 para salir. Repite si

	el valor no es un entero o está fuera del menú.
int leerCriterioBusqueda()	Pide elegir búsqueda por ID o por nombre y valida la opción.
String leerTexto(String solicitud)	Pide un texto y lo devuelve sin espacios en los extremos. Repite si está vacío.
void mostrarModulo(Modulo modulo)	Recibe cualquier tipo de módulo y muestra su información usando su toString().
void mostrarModulos(List<Modulo> modulos)	Recorre la lista y usa mostrarModulo con cada uno. Sirve para listado, catálogo y búsquedas.
void mostrarCiclo(List<ResultadoOperacion> resultados, RecursosMision recursos, long numeroCiclo)	Muestra qué hizo cada módulo (o su causa de inactividad) y los saldos finales.
void mostrarResumen(ResumenComunicaciones resumen)	Muestra todos los campos del informe y el ID, nombre y estación de cada antena empataada en el máximo.
void mostrarMensaje(String mensaje)	Avisos, resultados vacíos y despedida.

5.11 Clase Controlador

Tiene dos referencias private final, recibidas en el constructor: Mision mision y VistaConsola vista. No guarda copias de energía ni de datos.

- void iniciar(): mantiene el bucle del menú hasta que se elige salir. Cada opción se delega a un método privado y luego se vuelve al menú.
- Métodos privados: listarModulos(), buscarModulo(), mostrarCatalogo(), avanzarCiclo() y mostrarComunicaciones(). Cada uno pide datos a la vista, solicita la operación al modelo y muestra la respuesta. buscarModulo() convierte el Optional en una coincidencia o en un aviso.

- No hace cálculos de consumo ni de descarga.

5.12 Clase Main

Es el punto de entrada y no guarda estado de negocio. En `main(String[] args)` obtiene la misión inicial, crea `VistaConsola` y `Controlador` y llama a `iniciar()`.

`private static Mision crearMisionInicial()` construye los diez módulos de la sección 8, los mete en una `List<Modulo>` en el orden indicado y crea `RecursosMision` y `Mision`. Sus datos son solo configuración: las validaciones las hacen los constructores y el modelo.

6. Herencia y polimorfismo

Herencia

`ModuloVuelo`, `ModuloTierra` y `ModuloEnergia` **son** un `Modulo` (relación de generalización). Cada uno tiene ID, nombre, salud, actividad y costo, y se puede usar en cualquier lugar donde se espere un `Modulo`. La superclase guarda lo común; las subclases agregan sus datos y su algoritmo propio.

`Mision` **tiene** módulos y **usa** `RecursosMision`. Esas son asociaciones, no herencia.

Polimorfismo en parámetros y referencias

- `Mision` recibe una `List<Modulo>` en su constructor.
- `mostrarModulos` recibe `List<Modulo>`.
- `mostrarModulo(Modulo modulo)` acepta cualquiera de los tres tipos.

El contrato de colaboración se expresa mediante un `for`.

La variable `modulo` es de tipo `Modulo`, pero Java ejecuta la versión del método que corresponde al tipo real del objeto.

Sobrescritura de métodos

Método	Dónde se declara o sobrescribe	Para qué
procesarCiclo	Abstracto en <code>Modulo</code> ; <code>@Override</code> en las tres subclases.	Aplicar la operación propia de cada tipo, con la misma firma y el mismo tipo de retorno.
toString	<code>@Override</code> de <code>Object</code> en <code>Modulo</code> ; <code>@Override</code> otra vez en las tres subclases.	Mostrar los atributos comunes y los propios.
equals y hashCode	<code>@Override</code> de <code>Object</code> en <code>Modulo</code> ; final, así las subclases no los cambian.	Mantener una identidad uniforme, basada en el ID.

Decisión sobre la firma de procesarCiclo

Criterio de diseño para procesarCiclo: Se determinó pasar la instancia de RecursosMision como parámetro explícito (procesarCiclo(RecursosMision recursos)). De esta forma se garantiza el paso de mensajes entre objetos sin forzar a que cada módulo mantenga un atributo interno redundante con la referencia a los recursos globales.

Principios que sostienen el diseño

- **Encapsulación:** nadie modifica los saldos sin pasar por las validaciones de RecursosMision.
- **Responsabilidad única:** una clase opera, otra coordina la misión y otra presenta la información.
- **Sustitución:** todas las subclases cumplen el mismo contrato, así que cualquiera puede usarse donde se espera un Modulo.

7. Flujos funcionales

Listar y buscar

Para listar, el controlador pide listarModulos() y entrega la lista a la vista. Para buscar, la vista obtiene el criterio y el texto, y el controlador llama a buscarPorId o buscarPorNombre. El resultado se muestra con la misma operación de presentación. Si no hay texto o no hay coincidencias, se muestra un aviso y se vuelve al menú sin modificar la misión.

Catálogo por costo

Mision copia las referencias de la lista principal, ordena la copia por costo (con desempate por ID) y la entrega a la vista. El orden original no cambia, así que consultar el catálogo no afecta al siguiente ciclo.

Avanzar un ciclo

1. El controlador pide avanzarCiclo(); la misión prepara una lista de resultados vacía.
2. La misión recorre los módulos en el orden de carga y llama a su operación polimórfica con la **misma** instancia de RecursosMision.
3. Cada módulo comprueba sus condiciones. Si puede operar, valida los cálculos, actualiza los recursos y su acumulado. Si no, devuelve la causa sin cambios.
4. La misión guarda el resultado y sigue con el siguiente módulo (sin reiniciar recursos ni dar segundo turno).
5. Al terminar el recorrido, sube el contador de ciclos y devuelve los resultados. La vista muestra lo que hizo cada módulo y el estado global final.

La actualización de recursos y la del acumulado del módulo van juntas. Antes de cambiar nada se comprueban los límites numéricos de ambas, para que un número demasiado grande no deje los recursos actualizados sin su histórico.

Resumen de comunicaciones

Mision recorre la lista e identifica los módulos de tierra. Cuenta todos y los activos; suma la capacidad solo de los activos y el histórico de todos, aunque estén inactivos. Para los máximos, guarda el mayor histórico visto y una lista de empates: si encuentra uno mayor, reemplaza la lista; si encuentra uno igual, agrega la antena.

La capacidad activa es un máximo teórico por ciclo: no garantiza una descarga real de esa cantidad, porque puede faltar energía o datos. El total calculado debe coincidir con `RecursosMision.totalDescargado`.

Salida y errores

- Elegir **0** termina el bucle del menú.
- Los errores de formato del operador se manejan en la vista, que vuelve a pedir el dato.
- Una configuración inválida se rechaza **antes** de mostrar el menú y no se confunde con un impedimento normal de un módulo.

8. Carga inicial propuesta

Se cargan diez módulos después de **tres ciclos completos**

Orden	ID	Nombre	Tipo	Activo	Salud	Costo
1	E01	Panel Alfa	Energía	Sí	100	1000
2	V01	Cámara Norte	Vuelo	Sí	100	2500
3	T01	Antena Campus	Tierra	Sí	100	1800
4	E02	Panel Beta	Energía	Sí	100	1200
5	V02	Sensor Térmico	Vuelo	Sí	100	1600
6	T02	Antena Laboratorio	Tierra	Sí	100	2100
7	V03	Cámara Sur	Vuelo	Sí	100	2400
8	E03	Panel Reserva	Energía	No	50	1400
9	V04	Sensor Respaldo	Vuelo	No	45	1500
10	T03	Antena Auxiliar	Tierra	Sí	100	1700

Módulos de energía

ID	Genera por ciclo	Ciclos	Total generado
----	------------------	--------	----------------

E01	20	3	60
E02	12	3	36
E03	15	0	0

Módulos de vuelo

ID	Instrumento	MB por ciclo	Consumo	Ciclos	Total recolectado (MB)
V01	CAMARA	15	8	3	45
V02	SENSOR	8	4	3	24
V03	CAMARA	10	6	3	30
V04	SENSOR	6	3	0	0

Módulos de tierra

ID	Estación	Capacidad (MB)	Consumo	Total descargado (MB)
T01	Campus Central	10	5	30
T02	Laboratorio	10	6	30
T03	Campus Norte	8	4	24

Estado de los recursos al cargar

7 unidades de energía, **15** MB pendientes en órbita y **84** MB descargados. El contador de la misión vale **3**.

Coherencia histórica

Estos valores se alcanzan ejecutando tres ciclos con los parámetros anteriores, empezando con 10 unidades de energía y sin datos:

- Energía: E01 y E02 generan 96 unidades en total (32 por ciclo).
- Vuelo consume 54 (18 por ciclo) y tierra consume 45 (15 por ciclo). Entonces $10 + 96 - 54 - 45 = 7$.
- Vuelo recolecta 99 MB, se descargan 84 y quedan **15** en órbita.
- Los módulos inactivos (E03 y V04) tienen cero ciclos.

Resumen inicial: 3 antenas, todas activas, con capacidad conjunta de 28 MB por ciclo. El histórico total es 84 MB y el máximo es 30 MB, que comparten T01 (Antena Campus, estación Campus Central) y T02 (Antena Laboratorio, estación Laboratorio).

Resultado esperado del cuarto ciclo

Paso	Operación	Energía	Órbita (MB)	Descargado (MB)
Inicio	Estado cargado	7	15	84
E01	Genera 20 unidades	27	15	84
V01	Consume 8 y recolecta 15 MB	19	30	84
T01	Consume 5 y descarga 10 MB	14	20	94
E02	Genera 12 unidades	26	20	94
V02	Consume 4 y recolecta 8 MB	22	28	94
T02	Consume 6 y descarga 10 MB	16	18	104
V03	Consume 6 y recolecta 10 MB	10	28	104
E03	Inactivo, sin cambios	10	28	104
V04	Inactivo, sin cambios	10	28	104
T03	Consume 4 y descarga 8 MB	6	20	112

Al terminar, ciclos Transcurridos vale **4**. E01, E02, V01, V02 y V03 tienen 4 ciclos efectivos; E03 y V04 siguen en 0. Los históricos de T01 y T02 son 40 MB cada uno y el de T03 es 32 MB, así que el empate del máximo se mantiene. El quinto ciclo empieza con **6** unidades de energía, **20** MB en órbita y **112** MB descargados.

Reglas que siempre deben cumplirse

- El histórico global descargado es igual a la suma de los históricos de las antenas.
- Total recolectado = MB en órbita + MB descargados
- La lista conserva siempre el mismo orden.
- Los recursos nunca son negativos.
- Los ciclos acumulados de un módulo nunca superan los ciclos transcurridos de la misión.

9. Validaciones y casos de aceptación

Posibles resultados que se esperan:

Caso	Condición o acción	Resultado esperado
------	--------------------	--------------------

CA01	Cargar la misión propuesta.	Diez módulos, los tres tipos, y los saldos e históricos de la sección 8. IDs únicos y acumulados coherentes.
CA02	Vuelo necesita 8 unidades y hay exactamente 8.	Recolecta toda su capacidad, la energía queda en 0 y su contador sube una vez.
CA03	Vuelo necesita 8 unidades y hay 7.	No consume ni recolecta, su contador no sube y devuelve ENERGIA_INSUFICIENTE.
CA04	Cualquier módulo está inactivo.	Devuelve INACTIVO, no cambia recursos ni históricos y el siguiente módulo participa.
CA05	Tierra: capacidad 10, consumo 5, hay 3 MB y energía 5.	Descarga 3 MB, consume las 5 unidades y suma 3 a ambos históricos.
CA06	Tierra activa y no hay MB en órbita.	Devuelve SIN_DATOS y no consume energía, aunque le sobre.
CA07	Tierra encuentra datos pero no le alcanza la energía.	Devuelve ENERGIA_INSUFICIENTE; no hay descarga parcial por falta de energía.
CA08	Consultar el catálogo y luego avanzar.	El catálogo sale ordenado, pero el ciclo sigue el orden de la carga.
CA09	Consultar el resumen inicial.	Tres antenas activas, capacidad 28, histórico 84 y máximos T01 y T02.
CA10	Buscar un nombre repetido y un ID que no existe.	El nombre devuelve todas sus coincidencias; el ID inexistente muestra un aviso sin cambiar el estado.
CA11	Ejecutar el cuarto ciclo y empezar el quinto.	El cuarto termina con energía 6, órbita 20 y descargado 112; esos valores son el inicio del quinto.
CA12	Todos los históricos de tierra valen cero.	Todas las antenas empatan en el máximo; una inactiva también cuenta como candidata.
CA13	Crear un ID duplicado, un saldo negativo o acumulados incompatibles.	La carga se rechaza antes de iniciar el menú, con una causa identificable.

Validaciones adicionales: textos obligatorios no vacíos, instrumento válido, salud entre 0 y 100, capacidades y consumos positivos, tipos no nulos y operaciones sin pasarse del máximo del long. Lo que escribe el operador no se acepta hasta que cumple el formato de la opción elegida.

10. Relaciones para el UML y cobertura de la rúbrica

Relaciones que debe mostrar el diagrama

Relación	Tipo y multiplicidad
Modulo con sus tres subclases	Generalización desde ModuloVuelo, ModuloTierra y ModuloEnergía hacia Modulo. La superclase y procesarCiclo son abstractos.
Mision con Modulo	Asociación navegable desde la misión hacia una colección ordenada de 10 o más módulos (10..*). Cada módulo pertenece a una misión.
Mision con RecursosMision	Asociación uno a uno: una misión usa una sola instancia de recursos durante toda la ejecución.
Modulo con RecursosMision	Dependencia por parámetro en procesarCiclo. El módulo no guarda su propia copia de los recursos.
Controlador con Mision y VistaConsola	Asociaciones: el controlador mantiene una referencia a una misión y a una vista.
Main con los demás componentes	Dependencias de creación y de arranque. No hereda de las clases del modelo.
Resultados y presentación	Dependencias por tipos de retorno y parámetros. ResumenComunicaciones referencia temporalmente de 0 a muchos módulos de tierra (0..*).

Diagrama UML:

